



MERRIMACK COLLEGE

CSC 6033

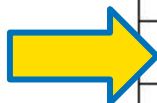
Week 4

Regular Expressions

Languages, Automata, and Decidability - Dr. Paulo Fernandes

Timeline

Where are we?



Unit	Week	Topic	Projects	Quizzes	In-class Exercises	Exam
1	1	From Python to C++	P#1	Q#1	E#1	
1	2	Developing with C++	P#2	Q#2	E#2	
2	3	Finite Automata	P#3	Q#3	E#3	
2	4	Regular Expressions	P#4	Q#4	E#4	
3	5	Pushdown Automata	P#5	Q#5	E#5	
4	6	Turing Machines	P#6	Q#6	E#6	
5	7	Processing Input and Output	P#7	Q#7	E#7	
6	8	Decidability				Final Exam (all units)



MERRIMACK COLLEGE

Don't let tasks pile up!

Presentation Agenda

Week 4

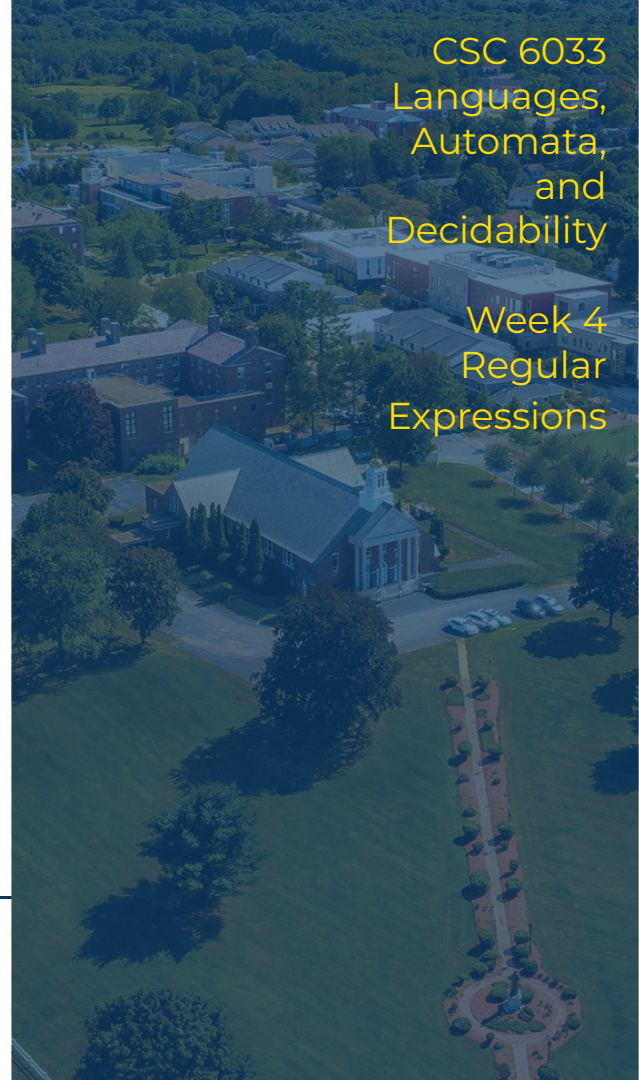
- Regular Expressions
 - a. Patterns and FSA
 - b. RegEx
 - c. Operations and Identities
- Equivalence with FSA
 - a. From FSA to RegEx
 - b. Limitations
 - c. From RegEx to FSA
- This Week's tasks



MERRIMACK COLLEGE

CSC 6033
Languages,
Automata,
and
Decidability

Week 4
Regular
Expressions



Regular Expressions

“Send me a postcard, drop me a line

Stating point of view

Indicate precisely what you mean to say

Yours sincerely, wasting away”

Paul McCartney (When I'm 64)

To define a language we can define the FSA that check it, state all words belonging to it (if this is a finite number), or conversely describe how the words of the language are.

However, the more usual way to define it is formally express in a regular expression how the words of the language can be generated.

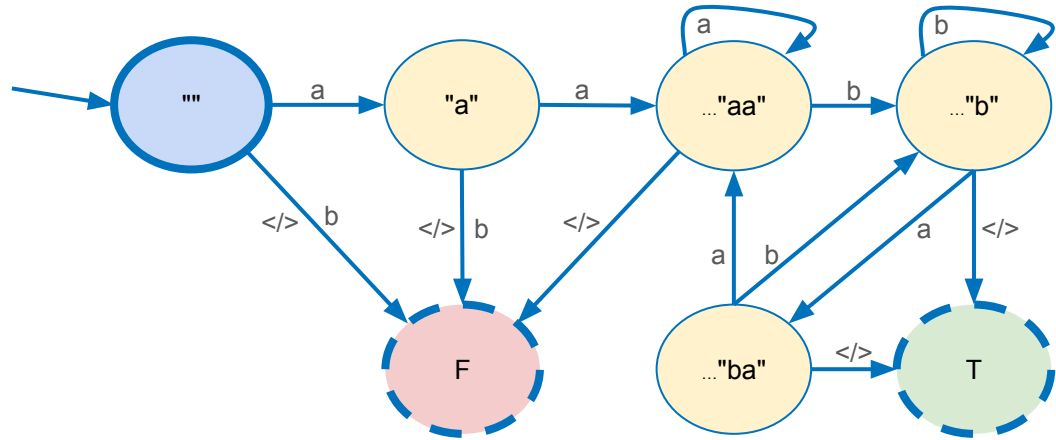
- **Patterns and FSA**
- RegEx
- Operations and Identities



Patterns and FSA

A Finite State Automaton defines a language, a set of words that may be accepted, and analogously a set of words that cannot be accepted.

For example,
considering the
alphabet $\Sigma = \{a, b\}$
what are the
acceptable words here
(the ones leading to
final state **T**)?

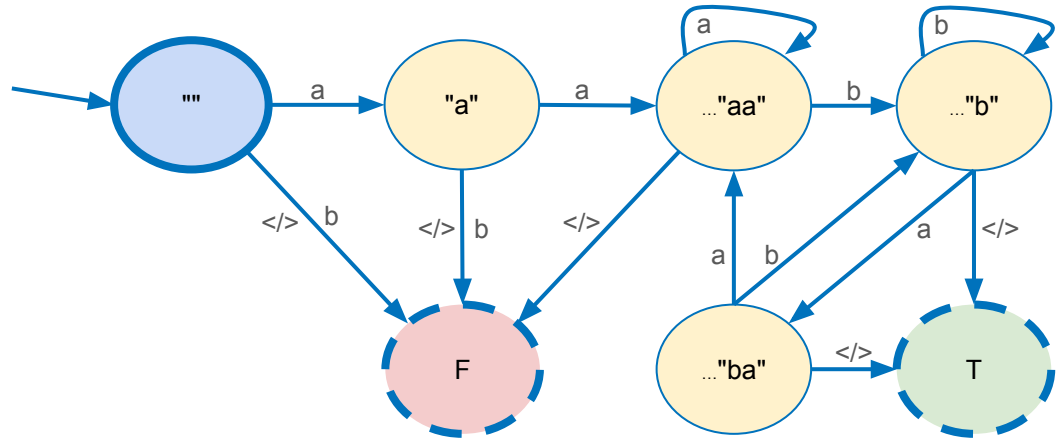


Patterns and FSA

Considering the alphabet $\Sigma = \{a, b\}$ the acceptable words here are words that start with "**aa**" and do not end with "**aa**" (if you remember last week examples).

However, just by seeing the FSA it would be more reasonable to express it as:

- words that start with "**aa**", then anything until
- end with "**ba**" or "**b**".

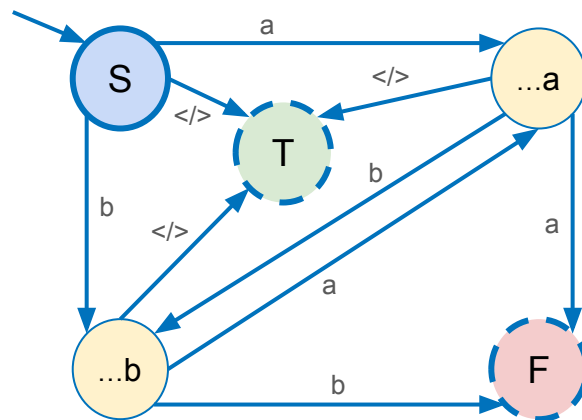


Patterns and FSA

Considering the alphabet $\Sigma = \{a, b\}$ what would be the acceptable words (leading to state **T**) for this FSA?

If you remember last week's example it is words without double letters:

- any sequence without "**aa**" or "**bb**".



In this case it could be easier to define unacceptable words as anything with either "**aa**" or "**bb**". Which is the complement of the acceptable words.



Regular Expressions

“Send me a postcard, drop me a line

Stating point of view

Indicate precisely what you mean to say

Yours sincerely, wasting away”

Paul McCartney (When I'm 64)

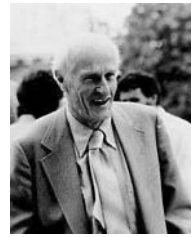
To define a language we can define the FSA that check it, state all words belonging to it (if this is a finite number), or conversely describe how the words of the language are.

However, the more usual way to define it is formally express in a regular expression how the words of the language can be generated.

- Patterns and FSA
- **RegEx**
- Operations and Identities

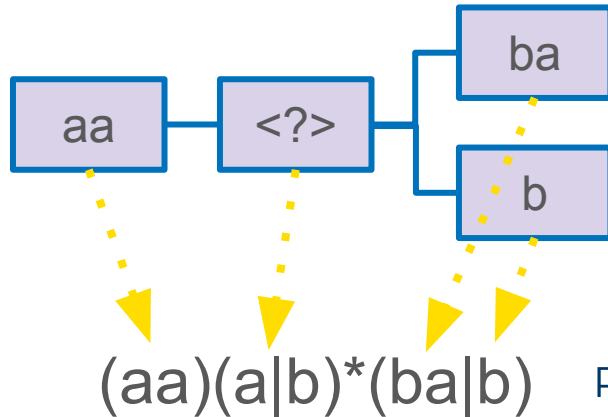


Regular Expressions

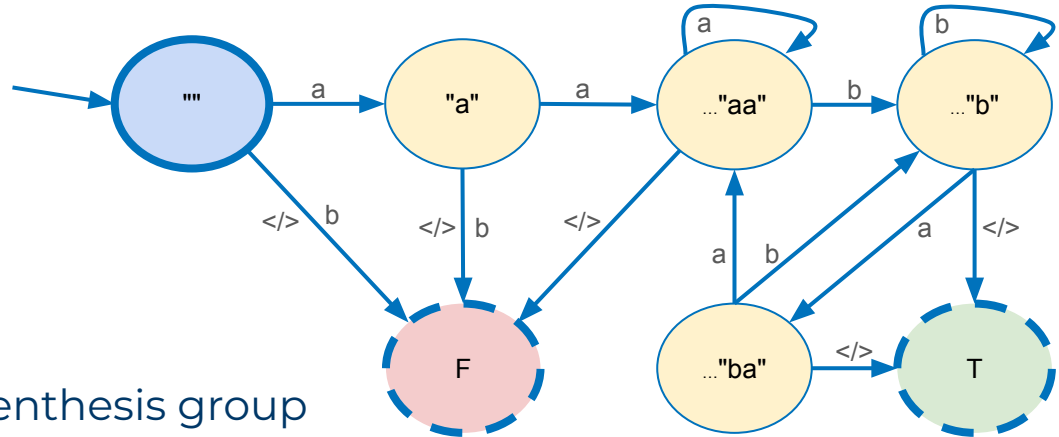


Stephen Cole Kleene

Words that start with "**aa**", then anything until end with "**ba**" or "**b**".



Parenthesis group



The symbol | means an or - The symbol * means zero or more of the preceding



Regular Expressions - RegEx

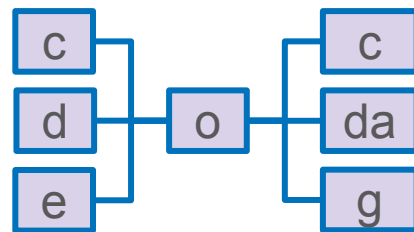
The basic elements of Regular expressions are (according to common formalisms usually found in many practical applications):

- two or more items together are a concatenation (including letters);
- the open and close parenthesis (...) are used to group parts of a RegEx;
- the symbol | represents an alternative between the items;
- the open and close brackets [...] serve to define a contiguous set of letters in which the letters are chosen alternatively;
 - For example, the RegEx **[c-e]o(c|da|g)** represents:

- **coc**
- **coda**
- **cog**

- **doc**
- **doda**
- **dog**

- **eoc**
- **eoda**
- **eog**



Regular Expressions - RegEx

The basic elements of Regular expressions are (according to common formalisms usually found in many practical applications):

- the symbol **?** represents zero or one occurrence of the preceding item;
- the symbol ***** represents zero or more occurrences of the preceding item;
- the symbol **+** represents one or more occurrences of the preceding item;
 - For example, the RegEx **[c-e]?o*(c|da|g)+** represents several words (actually infinite words), including:

- **c**
- **cc**
- **da**
- **ec**
- **dda**

- **oc**
- **dog**
- **doooooog**
- **ddac**
- **cogda**

- **dag**
- **cgg**
- **dg**
- **egda**
- **dcda**



Regular Expressions - RegEx

The basic elements of Regular expressions are (according to common formalisms usually found in many practical applications):

- the symbol ϵ represents an empty string ("");
- the symbol $.$ represents any letter of the alphabet (just one);
- the symbols $\{ n \}$ represents exactly n occurrences of the preceding item;
- the symbols $\{ n, m \}$ represents from n to m occurrences of the preceding item;
- the symbols $\{ n, \}$ represents from n or more occurrences of the preceding item;
- the symbols $\{ , m \}$ represents up to m occurrences of the preceding item;

For example,
with $\Sigma = \{ a, b \}$
the RegEx
 $\epsilon|b(a\{2\}.)^+$
includes:

- *empty*
- *baab*
- *baabaaaa*
- *baaa*
- *baaaaaab*



Regular Expressions

“Send me a postcard, drop me a line

Stating point of view

Indicate precisely what you mean to say

Yours sincerely, wasting away”

Paul McCartney (When I'm 64)

To define a language we can define the FSA that check it, state all words belonging to it (if this is a finite number), or conversely describe how the words of the language are.

However, the more usual way to define it is formally express in a regular expression how the words of the language can be generated.

- Patterns and FSA
- RegEx
- **Operations and Identities**



RegEx Operations

Let P and Q be RegEx (languages defined by a RegEx):

- Union: $P+Q$ means the union of P and Q :
 - if $P = \{ \text{ab}, \text{abb} \}$ and $Q = \{ \text{ba}, \text{baa} \}$
 - then $P+Q = \{ \text{ab}, \text{abb}, \text{ba}, \text{baa} \}$
- Concatenation: PQ means concatenation of P and Q :
 - if $P = \{ \text{ab}, \text{abb} \}$ and $Q = \{ \text{ba}, \text{baa} \}$
 - then $PQ = \{ \text{abba}, \text{abbaa}, \text{abbba}, \text{abbbaa} \}$
- Kleene Star: P^* means a RegEx that replicates P :
 - if $P = \{ \text{ab}, \text{abb} \}$
 - then $P^* = \{ \epsilon, \text{ab}, \text{abb}, \text{abab}, \text{ababb}, \text{abbab}, \text{abbabb}, \text{ababab}, \text{abababb}, \text{ababbab}, \dots \}$
- Pa means all words of P concatenated with the letter a ;
- ϵ means the words with no letters, thus we may also refer to ϵ as the RegEx that accepts only ϵ , thus $\epsilon = \{ \epsilon \}$;
- Φ means the RegEx that does not accept any word, thus $\Phi = \{ \}$.



RegEx Identities

Note that for operations over languages (RegEx) the symbol $+$ means the binary operator union, not the unary operator for letters for a repetition with at least one occurrence.

Let P , Q , and R be RegEx:

- $\Phi + P = P$
- $\Phi P + P\Phi = \Phi$
- $\epsilon P = P\epsilon = P$
- $\epsilon^* = \epsilon$
- $\Phi^* = \epsilon$
- $P + P = P$
- $P^*P^* = P^*$
- $PP^* = P^*P = P^+$
- $(P^*)^* = P^*$
- $(PQ)^*P = P(QP)^*$
- $(P+Q)^* = (P^*Q^*)^* = (P^*+Q^*)^*$
- $(P+Q)R = PR + QR$
- $R(P+Q) = RP + RQ$
- if $R = Q+RP$ and $\epsilon \notin P$ then $R = QP^*$



MERRIMACK COLLEGE

Video: [Identities of Regular Expression.](#)
Video: [Arden's Theorem.](#)

Equivalence with FSA

“Logic is the beginning of wisdom,
not the end.”

Leonard Nimoy

Both FSA and RegEx are capable of describing regular languages.

They are equivalent, and common logic derivation of regular expression formulations lead from FSA to RegEx and vice-versa.

It is not a simple process as it requires abstraction and search for alternatives.

- **From FSA to RegEx**
- Limitations
- From RegEx to FSA

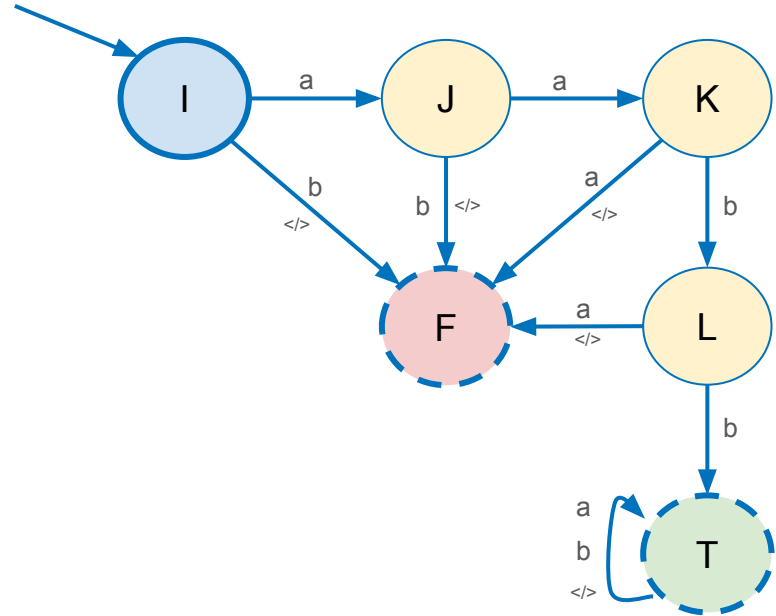


From FSA to RegEx

The process to generate a RegEx from a FSA is based on:

- Stripping the FSA to its simplest form;
- Associating a RegEx to each state;
- Apply the basic identities of RegEx to replace all state associated RegEx;
- Unify the RegEx of all final states.

All words starting with **aabb**.

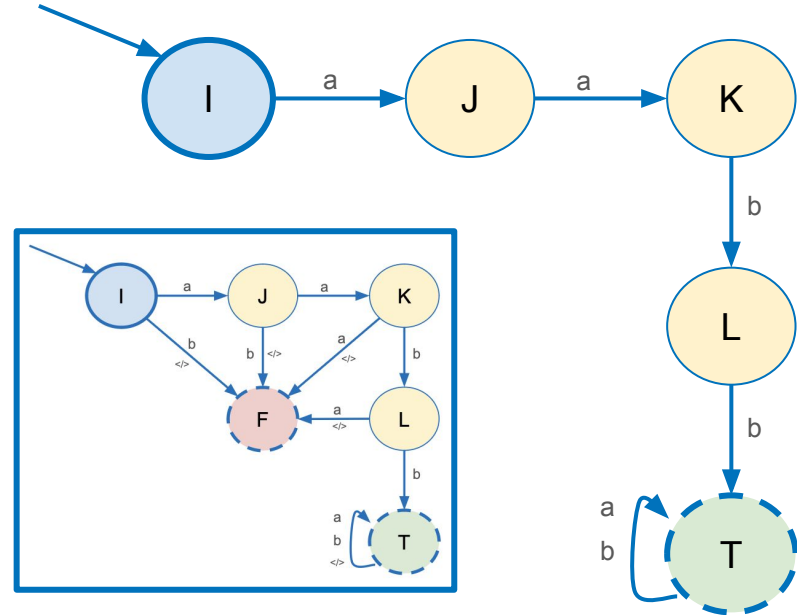


From FSA to RegEx

The process to generate a RegEx from a FSA is based on:

- **Stripping the FSA to its simplest form;**
- Associating a RegEx to each state by its inputting arcs;
- Apply the basic identities of RegEx to replace all state associated RegEx;
- Unify the RegEx of all final states.

All words starting with **aabb**.

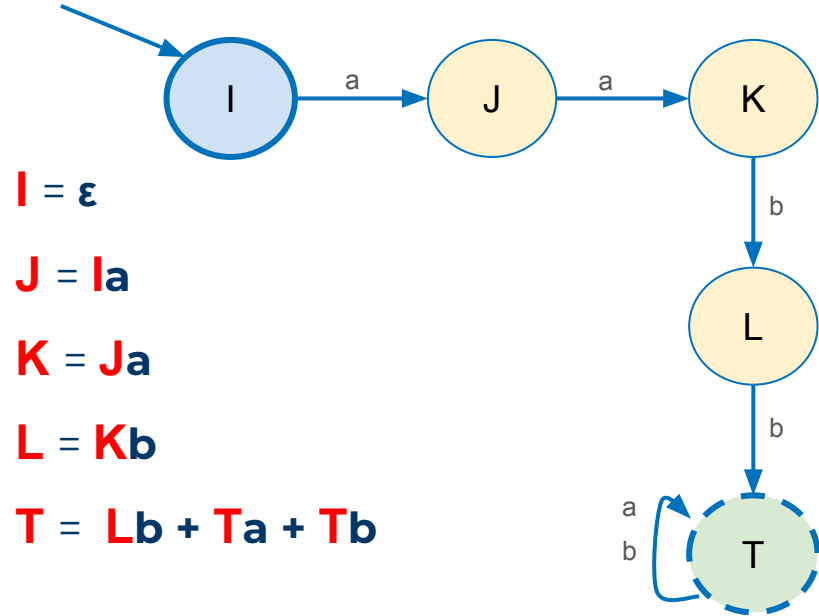


From FSA to RegEx

The process to generate a RegEx from a FSA is based on:

- Stripping the FSA to its simplest form;
- **Associating a RegEx to each state by its inputting arcs;**
- Apply the basic identities of RegEx to replace all state associated RegEx;
- Unify the RegEx of all final states.

All words starting with **aabb**.

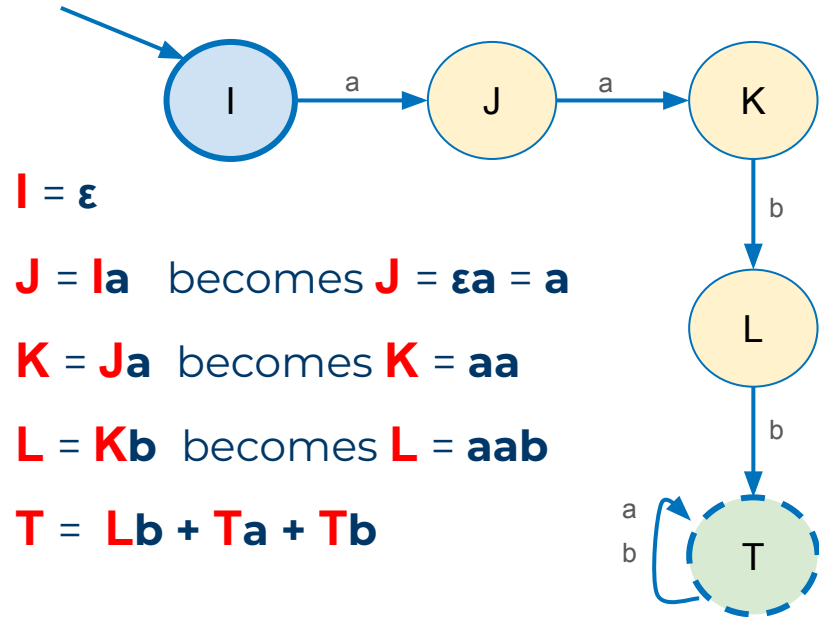


From FSA to RegEx

The process to generate a RegEx from a FSA is based on:

- Stripping the FSA to its simplest form;
- Associating a RegEx to each state by its inputting arcs;
- **Apply the basic identities of RegEx to replace all state associated RegEx;**
- Unify the RegEx of all final states.

All words starting with **aabb**.



MERRIMACK COLLEGE

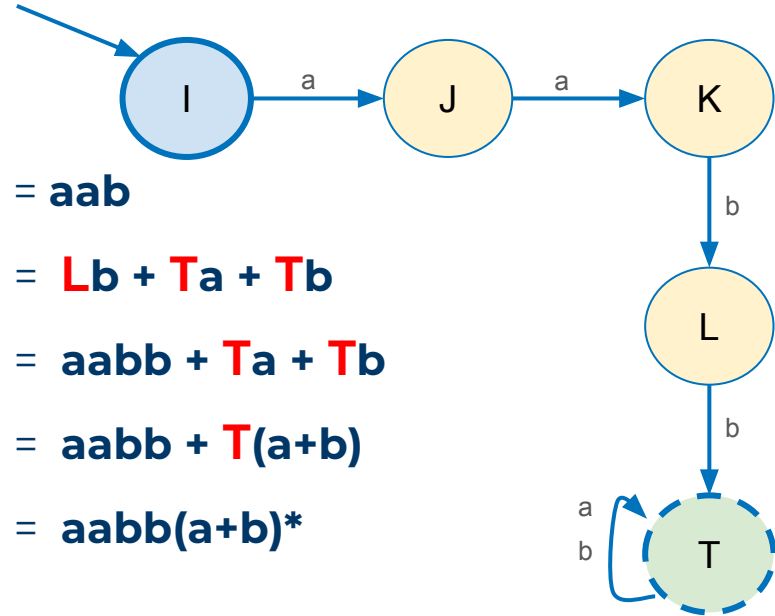
Replace **I** in **J** language. Apply $\epsilon P = P$.
Replace **J** in **K** language. Replace **K** in **L** language.

From FSA to RegEx

The process to generate a RegEx from a FSA is based on:

- Stripping the FSA to its simplest form;
- Associating a RegEx to each state by its inputting arcs;
- **Apply the basic identities of RegEx to replace all state associated RegEx;**
- Unify the RegEx of all final states.

All words starting with **aabb**.



$$L = aab$$

$$T = Lb + Ta + Tb$$

$$T = aabb + Ta + Tb$$

$$T = aabb + T(a+b)$$

$$T = aabb(a+b)^*$$



MERRIMACK COLLEGE

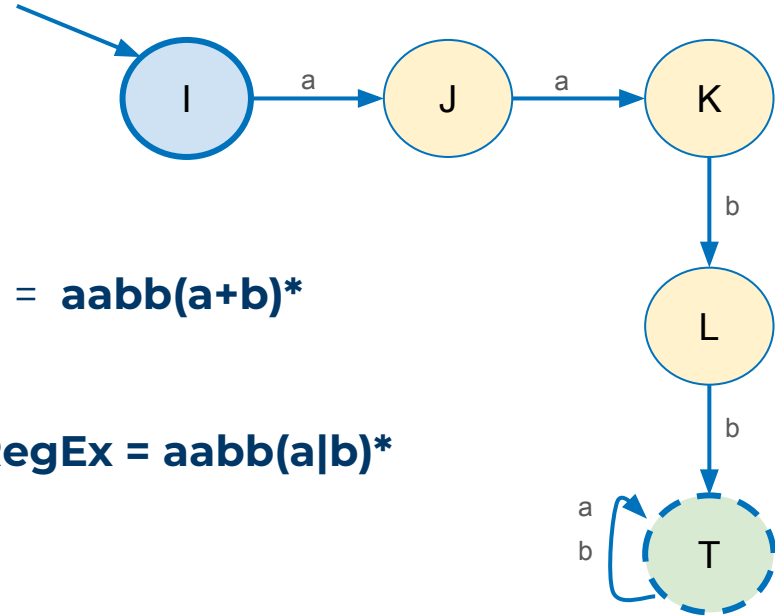
Replace **L** in **T** language. Apply **$R(P+Q) = RP + RQ$** .
Apply **$R = Q+RP$** and **$\epsilon \notin P$** then **$R = QP^*$** .

From FSA to RegEx

The process to generate a RegEx from a FSA is based on:

- Stripping the FSA to its simplest form;
- Associating a RegEx to each state by its inputting arcs;
- Apply the basic identities of RegEx to replace all state associated RegEx;
- **Unify the RegEx of all final states.**

All words starting with **aabb**.



T = **aabb(a+b)***

RegEx = **aabb(a|b)***



MERRIMACK COLLEGE

Video: [DFA to Regular Expression Conversion.](#)
Video: [when the DFA has Multiple Final States.](#)

Equivalence with FSA

“Logic is the beginning of wisdom,
not the end.”

Leonard Nimoy

Both FSA and RegEx are capable of describing regular languages.

They are equivalent, and common logic derivation of regular expression formulations lead from FSA to RegEx and vice-versa.

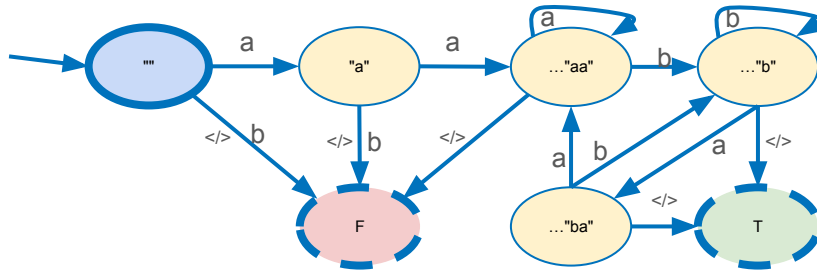
It is not a simple process as it requires abstraction and search for alternatives.

- From FSA to RegEx
- **Limitations**
- From RegEx to FSA



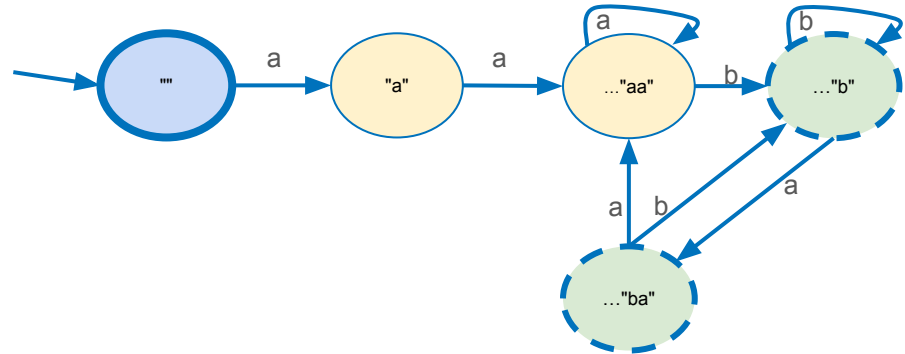
From FSA to RegEx - Limitations

Words that start with "**aa**", then anything until end with "**ba**" or "**b**".



Stripping the FSA to its simplest form:

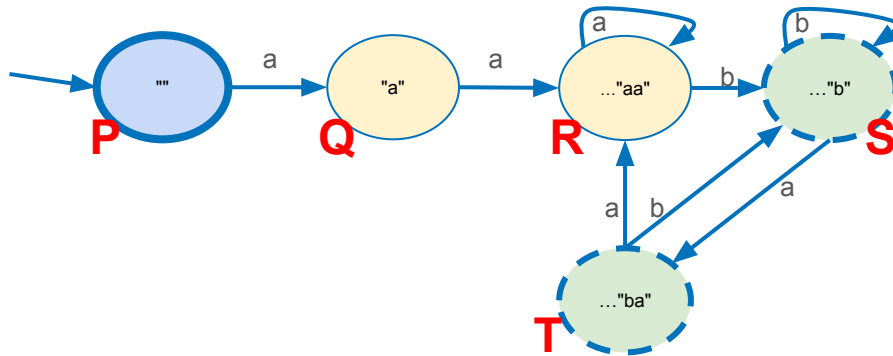
- Removing the complementary final state **F**;
- Removing the edges with end of word (**</>**).



From FSA to RegEx

Limitations

Words that start with "**aa**", then anything until end with "**ba**" or "**b**".



$$P = \epsilon$$

$$Q = Pa$$

$$R = Qa + Ra + Ta$$

$$S = Rb + Sb + Tb$$

$$T = Sa$$

$$S = Rb + Sb + Sab \quad \text{rep. T by Sa}$$

$$S = Rb + S(b+ab) \quad \text{by } S(Q+P) = SQ + SP$$

$$S = Rb(b+ab)^* \quad \text{by } S = Q+SP \rightarrow S = QP^*$$

$$Q = \epsilon a = a \quad \text{rep. P by } \epsilon \text{ and } \epsilon R = R$$

$$R = aa + Ra + Ta \quad \text{rep. Q by a}$$



MERRIMACK COLLEGE

Creating the languages of each state.
Applying the identities.

From FSA to RegEx - Limitations

Words that start with "**aa**", then anything until end with "**ba**" or "**b**".

$$P = \epsilon$$

$$Q = Pa$$

$$R = Qa + Ra + Ta$$

$$\boxed{S = Rb + Sb + Tb}$$

$$T = Sa$$

$$S = Rb(b+ab)^*$$

$$R = aa + Ra + Ta$$

$$R = aa + Ta + Ra$$

$$R = (aa + Ta)a^*$$

$$S = (aa + Ta)a^*b(b+ab)^*$$

$$S = (aa + Saa)a^*b(b+ab)^*$$

$$S = aaa^*b(b+ab)^* + Saaa^*b(b+ab)^* \quad \text{by } (Q+P)R = QR + PR$$

$$S = aaa^*b(b+ab)^*(aaa^*b(b+ab)^*)^* \quad \text{by } S = Q+SP \rightarrow S = QP^*$$

$$T = aaa^*b(b+ab)^*(aaa^*b(b+ab)^*)^*a \quad \text{rep. S on T by the previous definition}$$

rewriting R ($T+R = R+T$)

by $R = Q+RP \rightarrow R = QP^*$

rep. R on S by $(aa + Ta)a^*$

rep. T on S by Sa



MERRIMACK COLLEGE

RegEx = **S+T**: $aaa^*b(b|ab)^*(aaa^*b(b|ab)^*)^*(\epsilon|a)$
Note that a simpler form would be: $(aa)(a|b)^*(ba|b)$.

Equivalence with FSA

“Logic is the beginning of wisdom,
not the end.”

Leonard Nimoy

Both FSA and RegEx are capable of describing regular languages.

They are equivalent, and common logic derivation of regular expression formulations lead from FSA to RegEx and vice-versa.

It is not a simple process as it requires abstraction and search for alternatives.

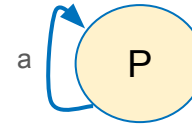
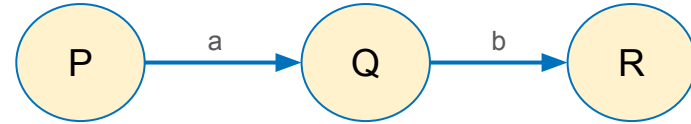
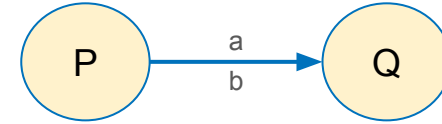
- From FSA to RegEx
- Limitations
- **From RegEx to FSA**



From RegEx to FSA

The process to generate a FSA from a RegEx is based on basic conversion rules:

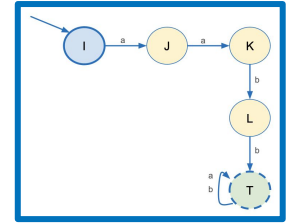
- An alternative **(a / b)** becomes:
- A concatenation **(ab)** becomes:
- A Kleene star **a*** becomes:



From RegEx to FSA

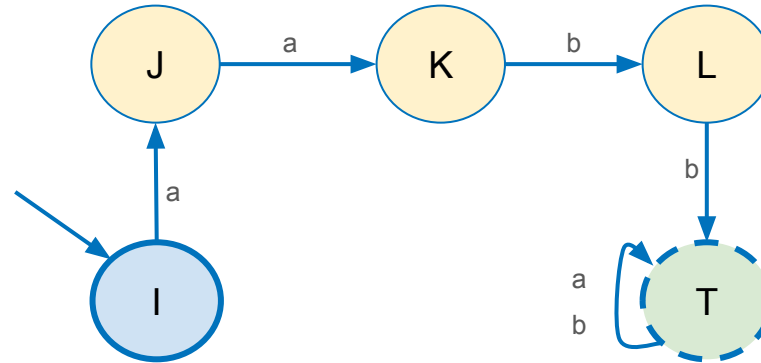
All words starting with **aabb**.

aabb(a+b)*



Applying the basic conversion rules:

- The start of the RegEx becomes the *initial state I*:
- The concatenation **aabb** becomes the states *I-J-K-L-T*:
- The Kleene star **(a/b)*** becomes the looped arc in *T*:
- The end of the expression becomes the *final state T*.



MERRIMACK COLLEGE

Video: [Conversion of RegEx to FSA - basics](#).
Video: [Conversion of RegEx to FSA - part 1](#) and [part 2](#).

This Week's tasks

- In-class Exercise E#4
- Coding Project P#4
- Quiz Q#4

Tasks

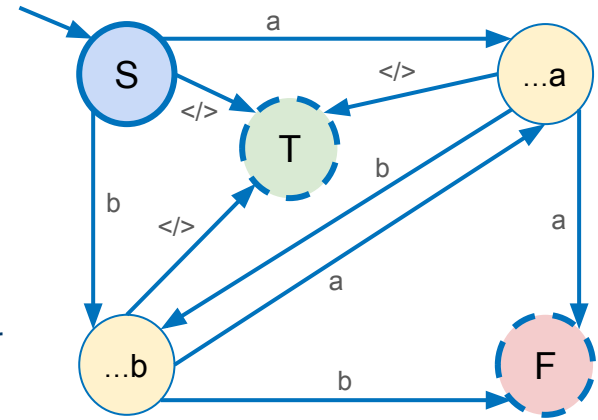
- Create a RegEx from a FSA.
- Create a C++ program to accept words of a language with a FSA created from a RegEx.
- Quiz #4 about this week topics.



In-class Exercise - E#4

Create the RegEx equivalent to the following DFA:

Your creation of the RegEx must be documented similarly in the examples presented in the slides 16 to 26, indicating each identity applied to obtain your RegEx.



Once your work is done and duly documented, save it into a **.pdf** file.

You have to submit the **.pdf** file with your RegEx and the process to obtain it on Canvas within the deadline.



MERRIMACK COLLEGE

This task counts towards the In-class Exercises grade and the deadline is This Saturday.

Fourth Coding Project - P#4

Create a DFA equivalent to the following RegEx:

$(a(a/b)^*) \mid (bb(a)^*)$

After that, build a C++ program with the definition of a class **FSA** that implements your FSA. Feel free to reuse or not your previous work of Week 3. Either way, the class should have at least:

- The enum for the states,
- A method for processing the chars,
- A method for deciding if the word is accepted or rejected.

Create also a **main** function that receives a string to be analyzed and delivers the result computed by the FSA.

Your task: Go to Canvas, and submit your C++ file (.cpp) within the deadline.



MERRIMACK COLLEGE

This assignment counts towards the Projects grade and the deadline is Next Tuesday.

Fourth Quiz - Q#4

- The fourth quiz in this course covers the topics of Week 4;
- The quiz will be available this Saturday, and it is composed by 10 questions;
- The quiz should be taken on Canvas (Module 4), and it is not a timed quiz:
 - You can take as long as you want to answer it;
- The quiz is open book, open notes, and you can even use any language Interpreter to answer it;
- Yet, the quiz is evaluated and you are allowed to submit it only once.

Your task:

- Go to Canvas, answer the quiz and submit it within the deadline.



MERRIMACK COLLEGE

This quiz counts towards the Quizzes grade and the deadline is Next Tuesday.

” Week 4 of CSC 6033

- Do In-class Exercise E#4 until Saturday;
- Do Quiz Q#4 (available Saturday) until next Tuesday;
- Do Coding Project P#4 until next Tuesday.

Next Week - Pushdown Automata



MERRIMACK COLLEGE